RingForge

A ring design studio with its own 3D geometry engine

Term Project Proposal · Open Source Software (Fall 2026)

Author	David (Student ID: _____)
Type	Individual project
Date	October 28, 2026
Deadline	December 6, 2026 (planned submission: December 5)
License	MIT

1. Summary

RingForge is an open-source web app for designing rings that can actually be made. You choose a size, a cross-section profile and a surface pattern, and RingForge shows the ring in 3D, calculates its weight and metal cost, and exports an STL file ready for 3D printing or wax casting.

Two things make it different from a simple "3D model generator":

1. **Everything geometric runs on my own engine.** Mesh generation, volume calculation, watertight checking, thickness analysis and the STL writer are written from scratch. No CAD library does the core work.
2. **It can merge two reference photos into a new design.** A user uploads two ring photos (for example from Pinterest), RingForge analyzes the design of each one, and merges them into a new ring that is a real, printable file in the user's size.

2. Motivation

- **Custom rings are popular, but hard to design.** Couple rings, name rings and fingerprint rings are common gifts, but designing one usually needs professional jewelry CAD software, which is expensive and takes a long time to learn.
- **Inspiration doesn't turn into a design file.** People save ring photos they like, but there is no simple way to go from "I like these two rings" to "here is a ring I can print."
- **Makers need numbers before casting.** Weight and cost depend on volume and metal density, and are tedious to estimate by hand.
- **General CAD tools don't know about jewelry.** They will happily export a ring with a 0.3 mm wall that cracks or fails to cast.

From a computer science point of view, this one real problem connects many topics: computational geometry, procedural generation, parsing, ray casting with acceleration structures, computer vision, and genetic algorithms.

3. Originality

I went through the full lists of term projects from 2021 to 2025 (**575 projects**):

- **0 projects** deal with jewelry, fabrication, or designing physical objects.
- Only about **6 projects** involve 3D graphics at all (e.g. *Blender object generator* 2021, *3D Icon Composer* 2025, *Vulkan Study* 2025), and none of them produce manufacturable models.
- RingForge avoids the most repeated categories: classic game clones (~35), chat bots (~34), to-do lists and timers (~31), and price prediction (~26).

Outside the class, existing options are either general-purpose CAD tools (OpenSCAD, build123d) that require programming or CAD skills, or professional jewelry CAD software. RingForge is open source, made specifically for rings, and adds features those tools don't have: photo-based design merging, interactive design evolution, and jewelry-specific manufacturability checks.

4. Features

#	Feature	What it does	CS technique
1	**Geometry engine** (core)	Turns a ring description into a watertight triangle mesh; computes exact volume; writes binary STL	Computational geometry, signed-tetrahedron volume, edge-manifold check
2	**Size & metal calculator**	Converts US / Korean (호) / Japanese / mm sizes to inner diameter; weight and cost from metal density	Unit modeling, input validation
3	**Pattern studio**	Grooves, hammered texture, Voronoi cells, waves, engraved Korean or English text	Procedural generation, Voronoi diagrams, text rasterization and surface mapping
4	**Image → ring**	Turns a fingerprint, drawing or photo into a relief on the ring	Image processing, brightness-to-depth mapping
5	**Ring recipe language**	A small text format (`.ring` files) that fully describes a design and can be shared or versioned with Git	Tokenizer, parser, interpreter, error reporting with line numbers
6	**Merge & evolve**	Combines two designs (union, morph, breed); interactive evolution where the user picks a favorite out of 6 children and breeds again	Genetic algorithms (crossover, mutation), interactive evolution
7	**Photo merge** (headline feature)	Two reference ring photos → each analyzed into a recipe → merged into a new printable ring	Segmentation, ellipse fitting, arcsin unwrapping + FFT, AI vision with output validation
8	**Thickness heatmap**	Colors the 3D preview red where the metal is thinner than a safe minimum (e.g. 0.8 mm)	Ray casting (Möller–Trumbore), BVH acceleration structure

#	Feature	What it does	CS technique
9	**Couple rings**	Two rings designed as a pair: text or pattern continues across both	Shared random seeds, boundary matching between height maps
10	**Size from photo**	Photo of a finger next to a credit card → estimated ring size	Edge detection, homography solved with SVD
11	**Mesh optimizer**	Reduces the triangle count of exported files	Quadric error metric simplification
12	**AR try-on** (experimental)	Webcam view shows the ring on the user's finger	Hand landmarks (MediaPipe) + my own 3D pose estimation and projection
13	**Web interface**	Tabs for each tool, live 3D preview, downloads	Gradio + three.js

Features 1–8 and 13 form the core of the project. Features 9–12 are planned too, but are built so that each one can be switched off without affecting the rest (see Section 10).

5. How It Works

5.1 The key idea: a ring is a height map

Unroll the outer surface of a ring into a flat grid: the angle **θ** goes around the ring, and **z** goes across its width. Every decoration is a function **h(θ, z)** that says how far the surface is pushed in or out at each point.

Example for a US size 7 ring:

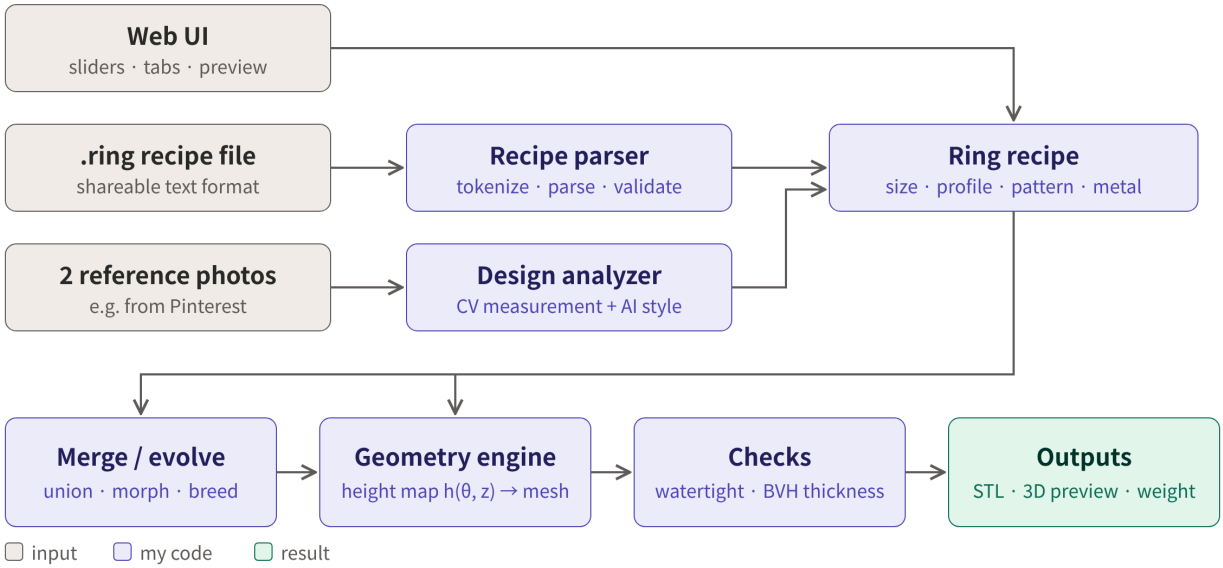
- Inner diameter: d = 11.63 + 0.8128 × 7 = **17.32 mm**
- Band: 4 mm wide, 1.5 mm thick → outer radius = 8.66 + 1.5 = 10.16 mm
- **Grooves:** h = −0.4 mm inside each of 24 slots, 0 elsewhere
- **Hammered:** 260 random shallow dents, up to 0.18 mm deep
- **Engraved text:** letters are drawn into an image, and each pixel's brightness becomes a depth

Because every design is just a function, **one engine powers every pattern**, and merging two designs means combining two functions:

- **Union** keeps every cut from both designs: h = min(h_A, h_B)
- **Morph** turns design A into design B around the ring, with weight a(θ) = (1 − cos θ) / 2

θ	0°	90°	180°	270°	360°
a(θ)	0 (pure A)	0.5	1 (pure B)	0.5	0 (back to A, no seam)

5.2 Architecture



All three ways of creating a design (the web UI, a `.ring` file, or two reference photos) produce the same thing: a **ring recipe**. Everything after that point is shared.

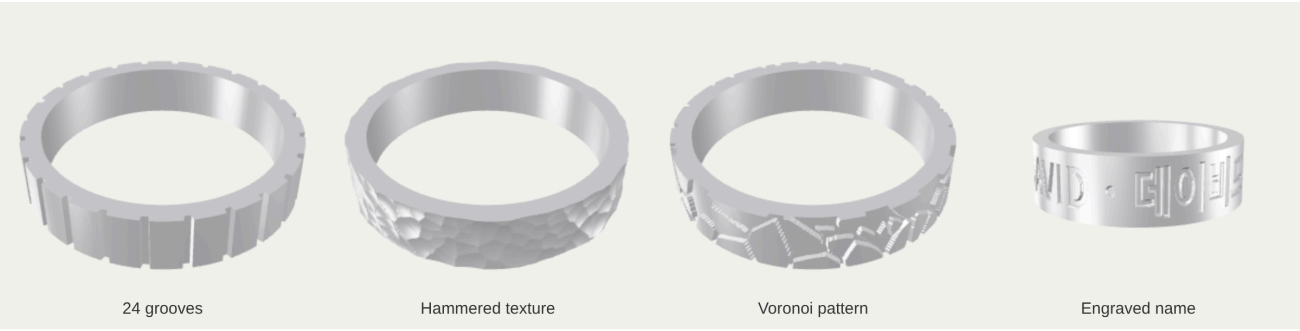
5.3 Photo merge pipeline

1. **Cut out the ring** from the photo background.
2. **Measure the shape.** The outer edge of a tilted ring appears as an ellipse. Example: ellipse width 620 px, visible band height 180 px → width ≈ 180 / 620 = 0.29 × outer diameter ≈ **5.9 mm** for a US 7 ring.
3. **Count the grooves.** Grooves near the edges of the photo look squeezed because the band curves away. Each point is mapped back to its true angle with θ = arcsin(x / R). Example: R = 310 px, x = 219 px → θ = **45°**. After this "unwrapping," an FFT finds the repeating pattern: 12 grooves across the visible half → about **24** around the whole ring.
4. **Name the style.** An AI vision model describes things that are hard to measure (finish, profile shape) and must answer in the recipe format. My parser validates the answer and rejects impossible values (e.g. a 3 mm groove on a 1.5 mm band).
5. **Merge** the two recipes (union, morph or breed).
6. **Build** the new ring in the user's size, check wall thickness, and show weight and cost.

Scope: RingForge designs bands with surface patterns (wedding bands, textured and engraved rings). Stones, prongs, twisted bands and through-holes are out of scope; for such photos the analyzer uses the band style only. Results are "inspired by" the references, not copies.

6. Prototype (Already Working)

To check that the idea is feasible, I built a prototype engine in about 150 lines of NumPy before writing this proposal.



24 grooves | Hammered texture | Voronoi pattern | Engraved name

Accuracy of the engine (plain band, US 7, 4 mm × 1.5 mm; exact volume = $\pi((r + t)^2 - r^2) \cdot w$ = **354.74 mm³**):

Segments	Triangles	Engine volume	Error	Watertight
16	128	345.69 mm³	2.55%	✓
64	512	354.17 mm³	0.16%	✓
256	2,048	354.71 mm³	0.010%	✓
720	5,760	354.74 mm³	0.0013%	✓

Decorated rings:

Design	Triangles	Volume	Weight (925 silver)	Build time
24 grooves	230,400	334.6 mm³	3.47 g	0.2 s
Hammered	129,600	321.3 mm³	3.33 g	0.3 s
Voronoi	230,400	324.8 mm³	3.37 g	0.3 s
Engraved text (6 mm band)	518,400	520.4 mm³	5.39 g	0.4 s

- Every mesh passed my own watertight check (every edge shared by exactly two triangles), and every volume matched the independent library **trimesh**.
- The grooved ring was also built with a completely different CAD engine (build123d): 335.6 mm³ vs. my 334.6 mm³, a difference of 0.3%.

Merge prototype (two parents, union, morph, and two bred children):



7. Tech Stack & Environment

Languages	Python 3.10+ (engine, analysis, server); JavaScript (3D viewer, AR page)
Runs in	A web browser. Locally: `pip install -r requirements.txt` → `python app.py` → `http://127.0.0.1:7860`. Online: hosted on Hugging Face Spaces, no installation needed
Platforms	Windows, macOS, Linux. No GPU required
Libraries	NumPy, Pillow, Gradio, three.js, OpenCV (image loading and basic filters), MediaPipe (AR only), an AI vision API (optional, photo merge only)
Quality	pytest test suite, GitHub Actions running tests on every push
Outputs	Binary STL (printing / casting), GLB (browser preview), `.ring` recipe files

8. My Work vs. External Code

Written by me	External (cited in README)
Geometry engine: mesh generation, volume, watertight check, STL writer	NumPy (arrays, math)
All pattern algorithms and text surface mapping	Pillow (drawing font glyphs into images)
Ring recipe language: tokenizer, parser, interpreter	Gradio, three.js (interface, 3D display)

Written by me	External (cited in README)
Merge modes and interactive evolution	OpenCV (image loading, basic filters)
Photo analyzer: ellipse fit, arcsin unwrapping, FFT groove counting, output validation	AI vision API (style description only)
BVH and thickness analysis, mesh optimizer	MediaPipe (hand landmarks for AR)
AR pose and projection math	trimesh (tests only, independent validation)
Interface design, tests, benchmarks	

AI assistance: this project is developed with the help of Claude (Anthropic) as a coding assistant. All AI-assisted work will be disclosed in the README. I review and test all code and will be able to explain every part of it.

9. Schedule

Week	Dates	Goals
1	Oct 29 – Nov 2	Repository setup, core engine, sizes and metals, tests, CI. Pattern studio: grooves, hammered, Voronoi, engraved text
2	Nov 3 – Nov 9	Recipe language, web interface with 3D preview, merge and evolution, couple rings, image → ring. Order a real 3D print
3	Nov 10 – Nov 16	Photo merge analyzer and accuracy test set, thickness heatmap (BVH), size from photo
4	Nov 17 – Nov 22	AR try-on (experimental), mesh optimizer, online deployment, full integration testing. **Feature freeze: Nov 22**
5–6	Nov 23 – Dec 5	Bug fixes only, README with GIF demos and benchmarks, photos of the real ring. **Submit: Dec 5**

10. Success Criteria

Goal	Target	Prototype so far
Volume accuracy ($N \geq 256$)	error < 0.01%	0.010% at N = 256
Watertight exports	100% of generated meshes (automated test)	12 of 12
Generation speed	< 1 s per ring	0.2 – 0.4 s
Thickness analysis speed-up	BVH $\geq 100\times$ faster than brute force	to be measured

Goal	Target	Prototype so far
Photo analyzer accuracy	groove count within ±2 on ≥ 80% of 20–30 hand-labeled photos; width within ±1 mm	to be measured
Real-world proof	at least one ring printed or cast from a RingForge file	planned for week 3

11. Risks & Mitigation

Risk	Mitigation
Too many features for the time available	Each feature is a separate module that can be switched off. Cut order if behind schedule: AR try-on → mesh optimizer → size from photo → couple rings. No new features after Nov 22
Photos of shiny metal are hard to analyze	Results are presented as "inspired by"; users can adjust everything manually; measured accuracy is reported honestly in the README
AI vision API needs a key and may be unavailable	The app fully works without it (measurement-only mode), so it always runs for evaluation
AR depends on webcam testing	Marked experimental; first feature to cut
Copyright of reference photos	User photos are never stored in the repository; demos use my own or freely licensed photos
Engine cannot make stones or through-holes	Declared out of scope; analyzer reports "band style only" for such photos

12. Deliverables

- Public GitHub repository and source code (zip) at submission time
- `README.md` with project description, GIF demos, screenshots, installation guide, benchmarks, accuracy results and references
- `LICENSE` file (MIT)
- Online demo link (Hugging Face Spaces)

13. References

- Möller, T., & Trumbore, B. (1997). Fast, minimum storage ray-triangle intersection. *Journal of Graphics Tools*, 2(1).
- Garland, M., & Heckbert, P. S. (1997). Surface simplification using quadric error metrics. *SIGGRAPH '97*.

- Fitzgibbon, A., Pilu, M., & Fisher, R. B. (1999). Direct least square fitting of ellipses. *IEEE TPAMI*, 21(5).
- Hartley, R., & Zisserman, A. (2004). *Multiple View Geometry in Computer Vision* (2nd ed.). Cambridge University Press.
- Dawkins, R. (1986). *The Blind Watchmaker* (Biomorphs, interactive evolution).
- Secretan, J., et al. (2008). Picbreeder: Evolving pictures collaboratively online. *CHI 2008*.
- Prior art: OpenSCAD, build123d (general-purpose open-source CAD).
- Libraries: NumPy, Pillow, Gradio, three.js, OpenCV, MediaPipe, trimesh, pytest.